Projet Informatique 1ère Année

PRO 3600

ChessRoyal

“Un jeu d'échec amélioré”

CHAMPION Solène

DEHAYE Arthur

LEHDILI Ilyès

SALVIGNOL Nathan

WEINSTEIN Liora

Enseignant responsable: Raddaoui Bradan

Date de soutenance: Vendredi 12 juin 2026



Table des matières

1 Introduction **4**
2 Cahier des charges **5**
2.1 Objectif et description du projet 5
2.2 Fonctionnalités attendues 5
2.3 Outils de développement 6
3 Développement **7**
3.1 Analyse du problème et spécification fonctionnelle 7
3.2 Conception préliminaire 8
3.2.1 Architecture réseau 8
3.2.2 Découpage en modules 8
3.2.3 Patron de conception et diagramme de classes 8
3.3 Conception détaillée 9
3.3.1 Modèle de données du plateau et déplacement des pièces 9
3.3.2 Système de cartes 9
3.3.3 Interface graphique 10
3.3.4 Protocole réseau 10
3.4 Codage 11
3.5 Tests unitaires 11
3.6 Tests d'intégration 11
3.7 Tests de validation 11
4 Interprétation des résultats **12**
4.1 Résultats 12
5 Manuel utilisateur **13**
5.1 Compilation et lancement de la partie 13
5.2 Connexion 13
5.3 Interface et règles de jeu 14
5.4 Présentation des cartes disponibles 15
6 Conclusion **17**
6.1 Bilan du projet 17
6.2 Perspectives d'approfondissement 17
6.3 Retours d'expérience personnel 18
Annexe A - Gestion de projet **20**
A.1 Planning prévisionnel 20
A.2 Plan de charge et suivit d'activité 20
Annexe B - Code source **22**

1 Introduction

Ce document présente la conception et la réalisation de ChessRoyal, un jeu d'échecs "amélioré", développé dans le cadre du projet informatique de première année à Télécom SudParis.

Le jeu d'échecs est un jeu de stratégie opposant deux joueurs sur un échiquier de 64 cases, dont les origines remontent au VIe siècle en Inde sous le nom de *chaturanga.* Le jeu se propage ensuite en Perse puis dans le monde arabe, avant d'atteindre l'Europe au Xe siècle. En Espagne et en Italie les règles du jeu évoluent vers celles que nous connaissons aujourd'hui. Dans ce jeu de réflexion, deux joueurs s'opposent en déplaçant leurs 16 pièces (8 pions, 2 tours, 2 cavaliers, 2 fous, 1 reine et 1 roi) et en mangeant les pièces adverses. La partie se termine alors par un "échec et mat" lorsque le roi adverse ne peut plus échapper la capture. De nos jours, les échecs comptent plus de 600 millions de joueurs à travers le monde et connaissent un regain de popularité considérable à l'ère du numérique, portée par les plateformes en lignes telles que Chess.com.



Ainsi nous nous sommes intéressés, à travers ce projet, à bousculer les règles originales du jeu en incorporant un système de cartes spéciales qui apportent des pouvoirs aux pièces ou qui modifient le fonctionnement usuel des cases du plateau, dans le but d'offrir une expérience de jeu à la fois fidèle à la tradition et résolument novatrice.

Après la présentation du cahier des charges, ce rapport détaille le développement du logiciel, de l'analyse fonctionnelle à la conception préliminaire et détaillée jusqu'aux tests. Enfin ce rapport dresse le manuel d'utilisation du jeu ainsi qu'un bilan critique des résultats obtenus et des perspectives d'évolution.

2 Cahier des charges

2.1 Objectif et description du projet

Dans le cadre du projet de développement informatique réalisé en groupe, nous avons choisi de coder un jeu d'échecs "amélioré", jouable en réseaux, s'appelant ChessRoyal.

Le jeu doit dans un premier temps proposer une version fonctionnelle des échecs respectant les règles classiques de déplacement, de capture et de coups spéciaux (roque, prise en passant, promotion..) des pièces. Nous souhaitions également implémenter une partie réseau au jeu, en offrant la possibilité aux joueurs de s'affronter à distance.

Enfin, et c'est là que le qualificatif "amélioré" trouve son sens, nous voulions ajouter au jeu classique d'échec un système de cartes spéciales, tirées aléatoirement par les joueurs, qui modifient certaines règles initiales. L'idée est donc de pouvoir modifier le comportement de certaines pièces ou l'aspect du plateau pour augmenter l'aspect stratégique du jeu en décuplant par la même occasion le champ des possibilités. Cet ajout vise ainsi à apporter une approche ludique et originale du jeu traditionnel.

2.2 Fonctionnalités attendues

Les fonctionnalités du projet se répartissent en quatre grands ensembles : les fonctionnalités du jeu d'échecs lui-même, celles liées au système de cartes, celles relatives au réseau, et enfin les fonctionnalités techniques d'interface. Les listes ci-dessous précisent, pour chaque ensemble, les éléments retenus.

Fonctionnalités principales du jeu d'échecs

- Déplacements légaux de chaque pièce
- Coups spéciaux : roque, prise en passant et promotion
- Gestion automatique de l'alternance des tours entre les joueurs

Fonctionnalités liées à l'affichage

- Affichage graphique du plateau d'échecs et des pièces
- Affichage des cases disponibles pour une pièce
- Interaction de l'utilisateur avec les pièces et la souris

Fonctionnalités réseau

- Connexion entre deux joueurs à distance
- Affichage du bon côté de l'échiquier et de la bonne couleur des pièces selon le joueur
- Transmission des coups joués
- Transmission des effets dus aux cartes

Fonctionnalités liées au système de cartes

- Système de pioche de défausse et de main des cartes
- Interface de visualisation de la pioche, de la défausse et des mains des joueurs (si une carte ne s'applique que sur un type de pièce et qu'il n'en reste plus sur le plateau)
- Activation des effets de chaque carte sur le plateau
- Cartes adverses non visibles

Fonctionnalités techniques

- Menu de fin de partie

Fonctionnalités non implémentées

- Horloge décomptant le temps restant pour chaque joueur
- L'affichage d'un score
- L'échec, l'échec et mat et le pat (calcul trop complexe une fois combiné aux cartes)

2.3 Outils de développement

ChessRoyal est codé intégralement en Java, tant pour la logique du jeu que pour son interface graphique, réalisée avec JavaFX. La partie réseaux repose sur une architecture client/serveur fondée sur des sockets. Les tests unitaires des déplacements de pièces sont écrits avec le framework JUnit. Enfin chaque membre du groupe utilise GitLab pour le travail collaboratif et le suivi des versions du code.

3 Développement

3.1 Analyse du problème et spécification fonctionnelle

Afin de concevoir ChessRoyal, nous avons commencé par décomposer le jeu en plusieurs éléments fonctionnels distincts :

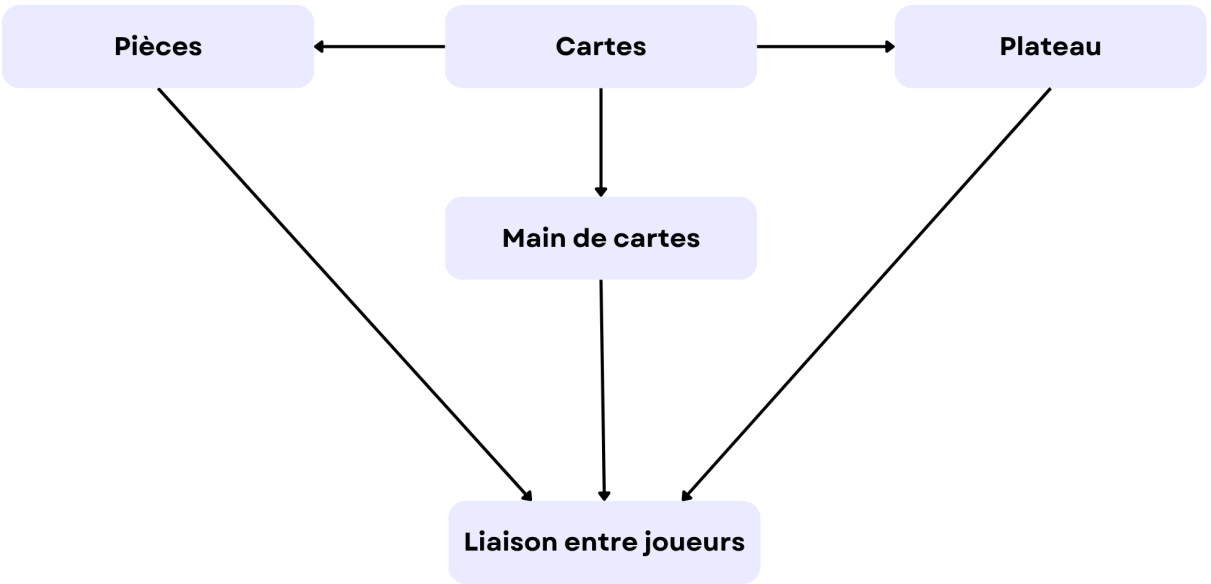
- **Les pièces** constituent les éléments actifs, ceux que le joueur manipule directement avec la souris. Chaque type de pièce obéit à des règles de déplacement qui lui sont propres et qu'il a fallu implémenter indépendamment les unes des autres.

- **Le plateau et ses cases** forment l'environnement de jeu. C'est une grille de 8x8 cases sur lesquelles les pièces évoluent. une case n'est pas nécessairement neutre, certaines peuvent acquérir un comportement particulier qui agit sur les pièces venant s'y poser.

- **Les cartes** représentent la couche stratégique propre à ChessRoyal. Elles sont tirées au cours de la partie et modifient le comportement d'une pièce ou l'état d'une case

- **La liaison entre les deux joueurs** doit enfin garantir que les deux instances du jeu partagent en permanence un état de partie identique. En effet, toute action décidée par un joueur doit pouvoir être répercutée fidèlement sur l'écran de son adversaire. Chaque joueur perçoit le plateau orienté de son propre côté et ne peut agir que sur ces pièces

Une fois que le problème eût été bien définis et les étapes de réalisation eut été bien organisée, nous nous sommes attaqués à la conception du jeu en codant chacun des éléments fonctionnels



3.2 Conception préliminaire

3.2.1 Architecture réseau

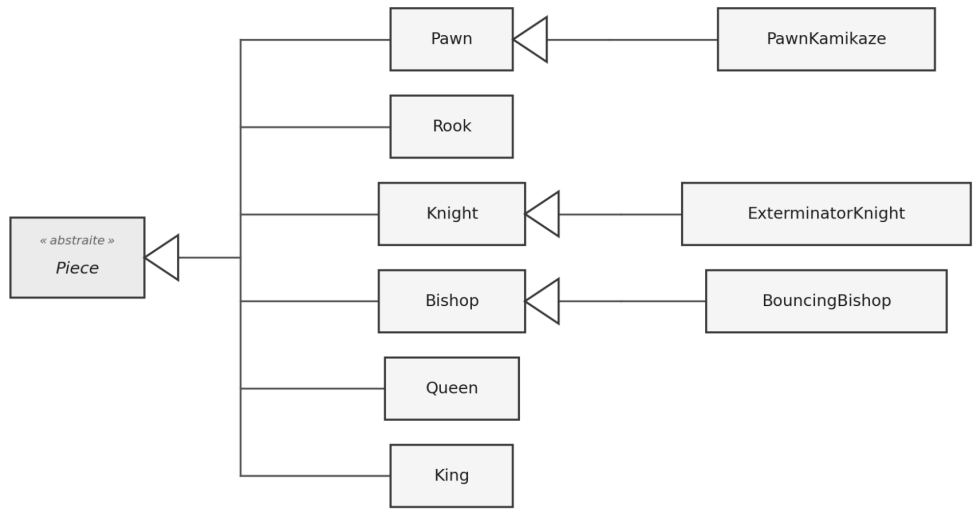
Le choix architectural clé est que le serveur n'est qu'un relais passif : il accueille les joueurs et redistribue les messages, sans jamais maintenir d'état de partie ni vérifier la légalité des coups. L'ensemble de la logique du jeu réside du côté client, dans le contrôleur ***ChessBoardCtrl.java*** et le package ***models***. La synchronisation est alors réalisée ainsi :
Le joueur actif applique une action en local (la légalité de l'action est vérifiée en local), si l'action est légale, elle est réalisée et émise côté serveur avant que le client adverse ne la reçoive et se contente seulement de rejouer cette action reçue de son côté.

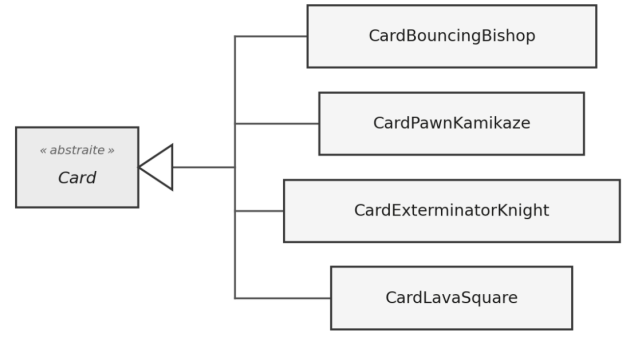
3.2.2 Découpage en modules

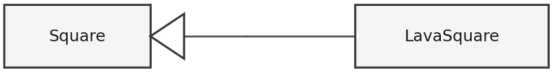
Le code se répartit en packages aux responsabilités distinctes. Le package ***models*** regroupe les règles de déplacement, la structure du plateau et les effets des cartes. Le package ***application*** gère le cycle de vie de JavaFX, les interactions de l'utilisateur et l'orchestration tour par tour. Le package ***message*** rassemble les objets sérialisables échangés ainsi que l'énumération du protocole. Le package ***réseau*** assure le transport TCP (Transmission Control Protocol).

3.2.3 Patron de conception et diagramme de classes

Le patron MVC (Modèle, Vue, Contrôleur) sépare le modèle (le tableau ***Square[][]*** et les classes de ***models***), la vue (les fichiers FXML et les images JavaFX) et les contrôleurs.
Dans le package ***models*** le code est hiérarchisé de la façon suivante :







3.3 Conception détaillée

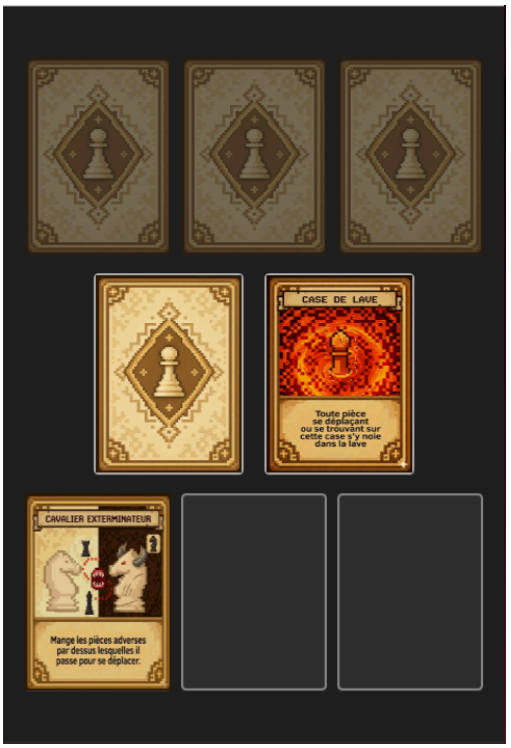
3.3.1 Modèle de données du plateau et déplacement des pièces

Le plateau est un tableau ***Square[][] board*** de taille 8x8. Chaque ***Square*** contient éventuellement une ***Piece***. Les coordonnées sont définies dans ***Position(row, col)***, et la couleur est encodée par un entier (0 pour les blancs et 1 pour les noirs). Chaque pièce mémorise sa position courante, sa position précédente (utile pour la prise en passant notamment) et l'indicateur signalant qu'elle vient de jouer.

La méthode commune ***moveTo()*** applique toujours le même algorithme : place la pièce sur la case d'arrivée, vider la case de départ, mettre à jour les positions et réinitialiser l'indicateur de dernier coup. L'étude des cases légales accessibles par une pièce est réalisée par la méthode ***getPossibleMoves()*** de la classe ***Piece*** et héritée par toutes ses classes filles. À noter que les pièces transformées par une carte sont toutes une nouvelle classe fille de ***Piece()*** et adapte ***getPossibleMove()*** selon leur effet.

3.3.2 Système de cartes

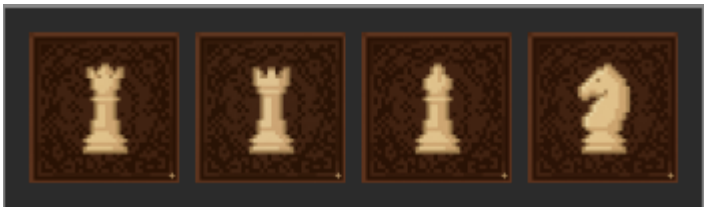
Le système de cartes repose sur une file de pioche, une main de trois emplacements et une case de défausse. La méthode ***apply(board, row, col)*** propre à chaque carte vérifie que la cible est bien la pièce ou la case attendue, la remplace si c'est valide, et reste sans effet sinon. La règle clé stipulant qu'un joueur ne peut effectuer qu'une action par tour, est garantie par le booléen ***aDejaAgi*** combiné à l'indicateur ***monTour***.



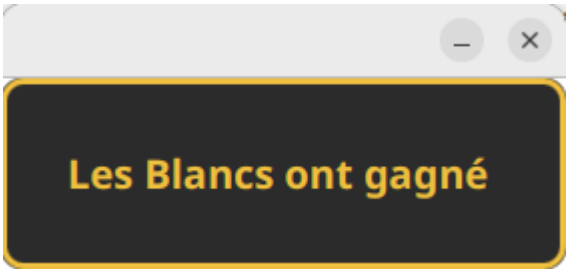
Ci-dessus nous avons une vue sur le système de cartes (main adverse en haut, pioche et défausse eu milieu et main du joueur en bas).

3.3.3 Interface graphique

À l'initialisation, le contrôleur ***ChessBoardCtrl*** construit le plateau, précharge les images des pièces, des cartes, et crée les 64 boutons de la grille qui forment l'échiquier. Le clic suit un ordre de priorité : si une pièce ou une carte est sélectionnée, cliquer à nouveau sur une case qui fait partie des coups possibles, applique le déplacement ou l'effet à ajouter. Sinon le second clic déselectionne la pièce/la carte, ou en sélectionne une nouvelle. La promotion ouvre une fenêtre bloquante proposant la dame, la tour, le fou ou le cavalier et remplace le pion promu.



La fin de partie est détectée par la recherche du roi de chaque couleur sur le plateau à chaque tour, sa disparition déclenche la fenêtre de victoire.



3.3.4 Protocole réseau

Tous les échanges transitent dans un objet ***Message*** encapsulant une donnée sérialisable et un type issu de l'énumération ***ProtocoleMessageType***. On distingue les messages de connexion, de demande de début de partie, d'attribution de numéro de joueur et les messages de jeu:

- ***COUP_ECHECS*** qui décrit un coup
- ***PRENDRE_CARTE*** qui signale à l'adversaire qu'une carte a été piochée.
- ***APPLIQUER_CARTE*** qui transmet le numéro de la carte et la case où elle s'applique

3.4 Codage

Le nommage des classes et des méthodes est mixte. Les classes sont en anglais (***ChessBoardCtrl***, ***BouncingBishop***,…) tandis que les méthodes et les commentaires sont en français (***deplacerPiece***, ***rafraichirVue***, …). Les images suivent une convention de nommage explicite combinant la pièce, la couleur, et le type de case.

3.5 Tests unitaires

Les tests unitaires visent uniquement le package ***models***, et plus précisément les méthodes de calcul des coups possibles et de déplacement des pièces standard, car ce sont les fonctions les plus importantes dans le jeu d'échecs et même pour ChessRoyal. Ils ont été écrits avec JUnit.
Chaque test construit un mini-plateau prédéfini, appelle la méthode de calcul des coups et compare les résultats à l'ensemble attendu, puis exécute un déplacement et vérifie la position alors obtenue. Il est alors possible de vérifier pour chaque pièce (dame, roi, pion, tour, cavalier, fou) le déplacement sur une case vide, le blocage par une pièce adverse, la capture d'une pièce adverse ou même des cas particuliers comme la prise en passant pour le pion ou le saut de pièce pour le cavalier. On vérifie également que la pièce ne va pas en dehors du plateau.

KnightTest.java

```java
package tests;

import models.Position;

/*
 * Tests pour la classe Knight
 */

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)

public class KnightTest {

    private Square[][] board = {{new Square(null), new Square(null), new Square(null), new Square(new Queen(new Position(0,3),0))},
                                {new Square(null), new Square(null), new Square(new Knight(new Position(1,2),1)), new Square(null)},
                                {new Square(new Pawn(new Position(2,0),1)), null, new Square(new Rook(new Position(2,2),1)), new Square(new Queen(new Position(2,3),1))},
                                {new Square(null), new Square(new Knight(new Position(3,1),0)), new Square(null), new Square(new Pawn(new Position(3,3),0))}
    };

    @Test
    @DisplayName("Test des cavaliers")
    @Order(1)
    public void Test() {
        //Test des déplacements possibles initiaux
        Assertions.assertArrayEquals(board[3][1].getPiece().getPossibleMoves(board), new Position[] {new Position(1,0), new Position(1,2), new Position(2,3)});
        Assertions.assertArrayEquals(board[1][2].getPiece().getPossibleMoves(board), new Position[] {new Position(3,1), new Position(3,3), new Position(0,0)});

        //Test de déplacement des caliers
        board[1][2].getPiece().moveTo(board,0,0);
        Assertions.assertEquals(null, board[1][2].getPiece());
        Assertions.assertEquals(0, board[0][0].getPiece().getRow());
        Assertions.assertEquals(0, board[0][0].getPiece().getCol());
        board[3][1].getPiece().moveTo(board,1, 2);

        //Second test de déplacment où le cavalier blanc mange le noir
        board[1][2].getPiece().moveTo(board,0,0);
        Assertions.assertEquals(null, board[1][2].getPiece());
        Assertions.assertEquals(0, board[0][0].getPiece().getRow());
        Assertions.assertEquals(0, board[0][0].getPiece().getCol());
        Assertions.assertEquals(0, board[0][0].getPiece().getColor());

        //Second test de déplacement possible où le cavalier blanc est bloqué
        board[0][3].getPiece().moveTo(board, 1, 2);
        Assertions.assertArrayEquals(board[0][0].getPiece().getPossibleMoves(board), new Position[] {});

    }


}
```

3.6 Tests d'intégration

Après validation des fonctions de déplacement, les modules ont été intégrés progressivement : couche réseau, interface graphique puis système de cartes. Les tests d'intégration ont donc porté sur la cohérence entre les 2 clients (le déplacement d'une pièce ou le fait de jouer une carte se répercute correctement sur le jeu de l'autre joueur) et sur l'enchaînement des tours grâce à l'indicateur : ***aDejaAgi***.

3.7 Tests de validation

Les tests de validation ont été menés en condition réelle pour vérifier que le jeu répond bien au cahier des charges.

Une première vérification a lieu lors du démarrage du serveur et de la connexion des deux joueurs à l'aide de pseudos distincts. Mais elle a également pour but de vérifier la bonne attribution des couleurs ainsi que le bon ordonnancement des tours entre les joueurs.

Une seconde vérification a porté sur la validité du système de carte : pioche, apparition d'un dos de carte chez l'adversaire, puis activation de chaque carte en observant si l'effet est bien celui voulu.

Une troisième vérification a été faite sur les règles spéciales implémentées, comme la promotion par exemple.

Enfin les derniers tests de validation ont été réalisés sur les règles que nous n'avons volontairement pas voulu implémenter, comme le fait qu'un joueur puisse exposer son propre roi sans que le coup soit refusé, ou qu'une position de mat ne termine pas la partie mais seulement la capture du roi.

4 Interprétation des résultats

4.1 Résultats

Le projet aboutit à un jeu fonctionnel réunissant les trois objectifs fixés. Le jeu d'échecs est jouable, avec l'ensemble des pièces et leurs déplacements, les principaux coups spéciaux (promotion, prise en passant, roque) et une fin de partie par capture du roi. La partie réseau du jeu fonctionne selon une architecture client/serveur, avec affichage correct de l'orientation et de la couleur selon le joueur. La synchronisation des coups, de la gestion des cartes ainsi que de leurs effets une fois jouée est également assurée par l'implémentation du réseau dans le jeu. Finalement l'élément distinctif du jeu, le système de carte, est totalement opérationnel et possède 4 cartes différentes qui agissent sur les pièces comme sur l'échiquier. Les cartes implémentées sont : Pion Kamikaze, Cavalier Exterminateur, Fou Bondissant et Lava Square.

Sur le plan des tests, les déplacements des pièces ont fait l'objet de tests unitaires JUnit, et l'ensemble a été validé en conditions réelles de partie à deux joueurs.

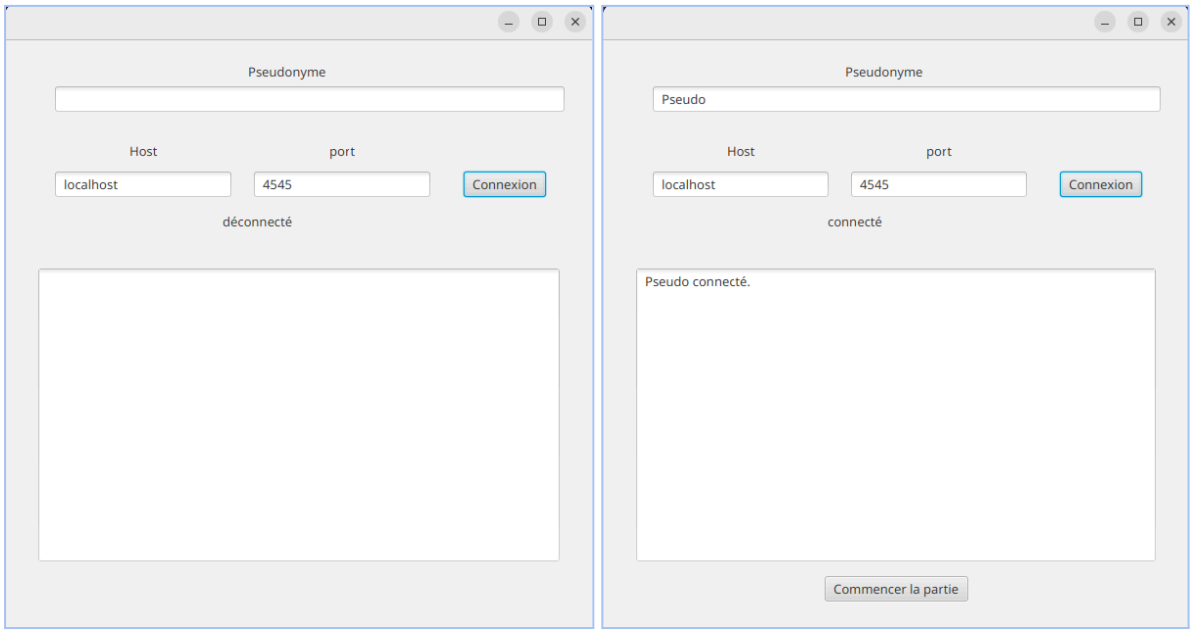
5 Manuel utilisateur

5.1 Compilation et lancement de la partie

Depuis un terminal ouvert à la racine du projet (le dossier ChessRoyal), la commande ***mvn package*** compile le projet et exécute les tests. Ensuite pour lancer une partie en réseaux, il suffit de lancer le serveur avec ***./Serveur.sh 4545*** dans la racine du projet (le port par défaut est 4545, mais tout autre port peut être choisi à condition que le serveur et les clients utilisent le même). Le serveur attend deux connexions avant de pouvoir lancer la partie. Chaque joueur démarre ensuite le client de son côté à l'aide de la commande ***./Client.sh*** toujours à la racine.

5.2 Connexion

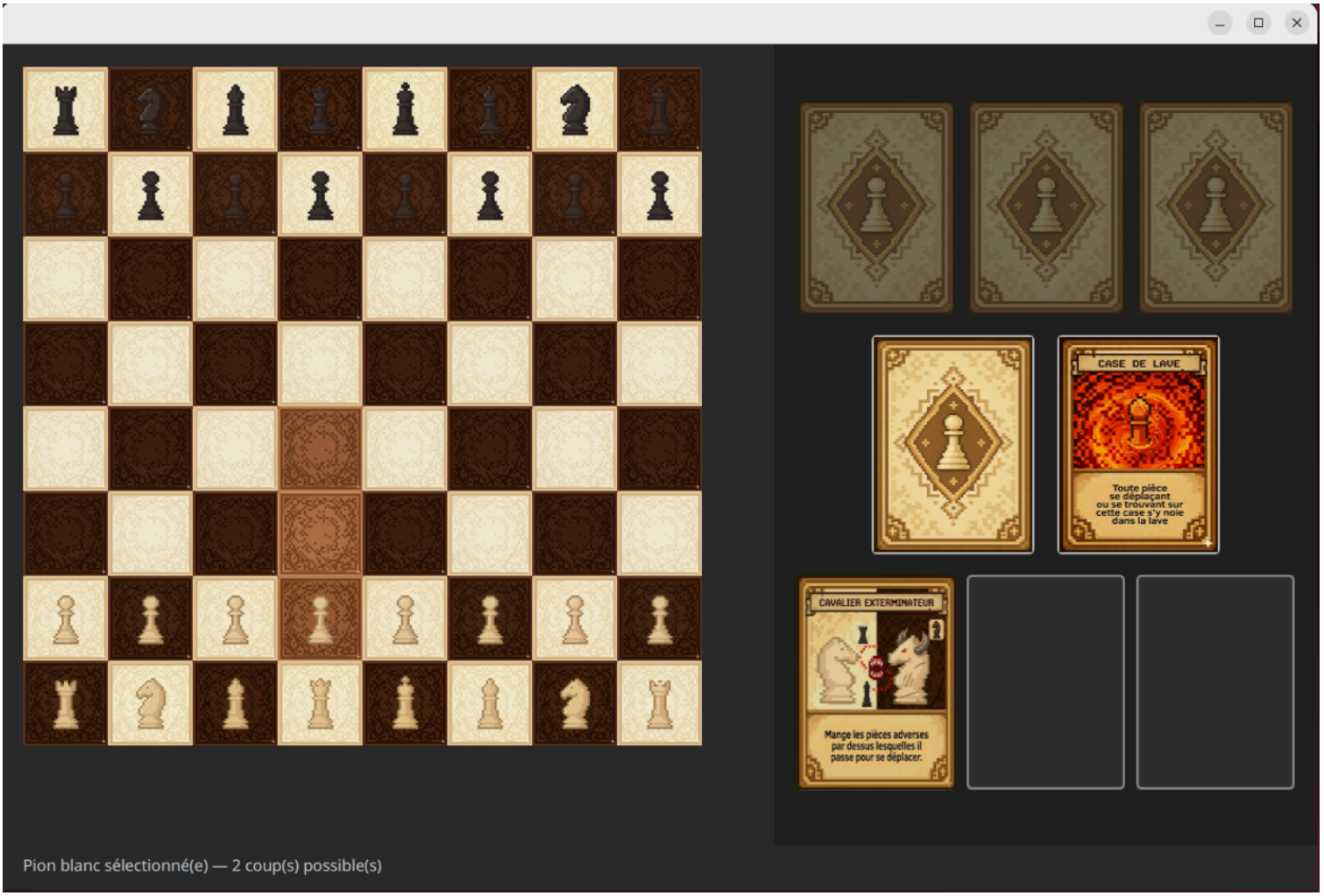
Au moment de la connexion, un menu apparaît et invite chaque joueur à renseigner: un pseudonyme, l'adresse du serveur (localhost par défaut si le client se trouve sur la même machine que le serveur) ainsi que le port. Une fois la connexion établie, le statut passe à "Connecté" et un bouton *Commencer la partie* devient disponible. Lorsque les deux joueurs sont listés, l'un deux lance la partie et les deux joueurs basculent automatiquement sur le plateau. Attention, les pièces blanches sont attribuées au premier joueur qui se connecte au serveur, et non pas à celui qui lance la partie.



5.3 Interface et règles de jeu

L'interface se compose du plateau 8×8 à gauche et, à droite, des cartes : celles de l'adversaire (dos visible) en haut, la pioche et la défausse au centre, et la main du joueur en bas. Une barre d'état affiche en permanence les messages de jeu. Pour déplacer une pièce, le joueur clique une première fois dessus pour la sélectionner et faire apparaître ses coups possibles, puis une seconde fois sur la case d'arrivée.

La règle essentielle est qu'à chaque tour, le joueur ne peut effectuer qu'une seule action : jouer un coup, piocher une carte ou jouer une carte. En cliquant sur la pioche on tire une carte et en cliquant sur une carte on la sélectionne. Pour défausser une carte il suffit de la sélectionner puis de cliquer sur la défausse ce qui ne consomme pas de tour. Enfin pour jouer une carte, il faut la sélectionner, puis l'appliquer sur la pièce ou la case voulue.

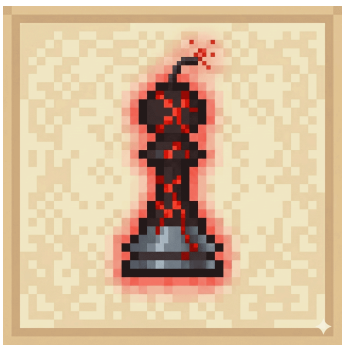


5.4 Présentation des cartes disponibles

Le jeu dispose d'une pioche composée de quatre cartes différentes (pouvant être tirée autant de fois que l'on souhaite). Chaque carte possède une description permettant de comprendre sont effet. Ces cartes sont:

- Le Pion Kamikaze







- Fou Bondissant







- Cavalier Exterminateur







- Case De Lave





6 Conclusion

6.1 Bilan du projet

Au travers de ce projet, nous avons pu mener à bien la conception complète d'une application, de la partie imagination / conception jusqu'à un logiciel pleinement jouable. Au-delà du résultat technique, ChessRoyal aura été l'opportunité pour nous de relever plusieurs défis formateurs, encore jamais abordés dans notre cursus scolaire, comme structurer une application de taille conséquente en modules cohérents ou programmer une communication réseau fiable entre deux clients. Mais la principale difficulté aura résidé dans le mélange entre règles traditionnelles des échecs et la liberté introduite par les cartes, dont les effets non conventionnels ont rendu certaines parties du code assez délicates à concevoir. Un exemple est que nous voulions originalement concevoir une carte qui ajoute une colonne/ligne complète à l'échiquier. Mais donc comment limiter ou augmenter le déplacement des pièces en conséquence (sans même mentionner les coups spéciaux comme la promotion) ?

Sur l'aspect de groupe, ce travail nous a appris à collaborer efficacement en équipe sur un projet de développement. Nous avons alors largement pu utiliser GitLab pour intégrer les contributions de chacun et mieux visionner l'avancement du projet. Si le jeu présente encore quelques bugs visuels et reste perfectible, nous sommes fiers d'être parvenus à un jeu fonctionnel en réseau, qui répond aux objectifs que nous nous étions fixés et qui constitue une base solide sur laquelle de nombreuses perspectives d'approfondissement peuvent émerger.

6.2 Perspectives d'approfondissement

Les perspectives d'approfondissement de ChessRoyal sont multiples et constituent autant de pistes stimulantes pour une suite du projet.
La première, et la plus logique, concerne l'implémentation des règles officielles du jeu d'échecs encore manquantes. Ce sont notamment la détection de l'échec, de l'échec et mat, le pat et l'interdiction des coups laissant le roi en position vulnérable. Ces fonctionnalités, pourtant fondamentales aux échecs, se sont révélées très complexes à intégrer dans ChessRoyal. En effet, les pièces ne sont plus les seuls éléments à analyser pour ces règles, il est également nécessaire de prendre en compte les effets des cartes. En partie par manque de temps, nous avons donc pris la décision de ne pas implémenter ces règles pour assurer la bonne conception des autres aspects du jeu.

Dans la continuité du système de cartes, une deuxième piste consisterait à enrichir la pioche par de nouvelles cartes aux effets variés (geler des pièces, ressusciter des pièces…). Cependant, l'ajout de nouvelles cartes soulève aussi la question de l'équilibre du jeu. Pour y répondre on pourrait envisager d'implémenter un système de pondération changeant les probabilités d'apparition au moment de tirer d'une carte. Une autre idée serait d'ajouter des critères supplémentaires aux cartes, les rendant indisponibles en début ou fin de partie.

Une troisième perspective serait l'ajout d'un adversaire artificiel permettant de jouer contre l'ordinateur, et donc d'un menu au début du jeu pour choisir les paramètres des parties.
Enfin, d'autres bonnes idées plus pratiques seraient l'ajout d'un chat en temps réel entre les deux joueurs ou encore l'ajout d'une horloge permettant d'afficher le temps de réflexion par coup.

Certaines de ces perspectives sont ambitieuses, mais restent raisonnables et pourraient réellement contribuer à enrichir l'expérience de jeu.

6.3 Retours d'expérience personnel

- Ilyes

"Ce projet a été pour moi une expérience particulièrement enrichissante et plaisante grâce à son côté ludique de pouvoir coder dans une application que l'on a pu imaginer. C'était notamment le premier projet de cette envergure auquel je participais et, au-delà de certaines connaissances techniques, il m'a surtout appris la méthodologie que l'on suit pour réaliser un projet de développement en équipe. Finalement ce qui m'a le plus plu est de voir le jeu prendre forme petit à petit."

- Liora

Le projet de développement informatique a été un des modules que j'ai le plus apprécié, notamment car il permet de développer et d'utiliser de manière concrète les compétences qu'on a pu développer au cours de l'année, plus précisément en Java pour ce projet. J'ai appris énormément de choses car j'ai eu la chance de toucher à beaucoup d'aspects du jeu. L'implémentation des pièces et des cartes n'ont pas nécessité beaucoup d'effort grâce au cours qu'on avait eu au début d'année. Cependant, pour la partie réseau et interface graphique, le tutoriel fourni par notre tuteur a été d'une grande aide. De plus, on est tous devenus à l'aise avec GitLab ce qui va nous être encore très utile dans le futur.

Au départ, je craignais qu'on ait du mal à concrétiser ce qu'on s'imaginait, surtout pour les cartes et le réseau, mais finalement tout s'est très bien passé et ce projet a été très formateur !

- Solène

"Ce projet à été très enrichissant pour moi, j'ai pu développer des connaissances en réseaux, qui me seront utiles dans la suite de mes études. De plus, par la nature de ce projet (la création d'un jeu) celui-ci était ludique et on a pu le voir prendre forme petit à petit. Finalement, c'est le premier projet d'une telle envergure que je réalise et il m'a permi de mieux appréhender les différentes étapes de la réalisation d'un projet de groupe, et de plus il m'a apporté des connaissances techniques mais surtout un cadre pour appliquer ce que nous avons vu en cours, ce qui m'a permis de mieux le comprendre."

- Nathan

Personnellement, j'ai toujours eu l'ambition de concevoir et de développer une application de A à Z. Ce module a donc représenté une opportunité particulièrement enrichissante, tant sur le plan des connaissances en développement et en programmation que sur celui du travail en équipe.

Le fait d'avoir pu intervenir sur l'ensemble des aspects du projet : de la logique du jeu à l'interface graphique, en passant par la partie réseau, m'a permis d'acquérir des compétences variées et complémentaires. Cette expérience m'a offert une vision globale du processus de développement d'une application et constitue une base solide pour les projets que je serai amené à réaliser à l'avenir.

Concernant ChessRoyal, il s'agit d'un projet auquel je suis particulièrement attaché. Son développement a suscité un réel intérêt de ma part, au point que je souhaite continuer à l'améliorer et à le faire évoluer au-delà du cadre de ce module.

- Arthur

Ce projet a été pour moi une expérience enrichissante. D'une part par la mise en œuvre de mes connaissances en Java, et l'apprentissage de nouvelles, pour créer toute une application fonctionnelle, et d'autre part par l'apprentissage du travail en équipe. La discussion et la coopération ont été des éléments essentiels pour assurer la concrétisation du projet, et ce sont des expériences qui ne manqueront pas d'être utiles dans le futur.

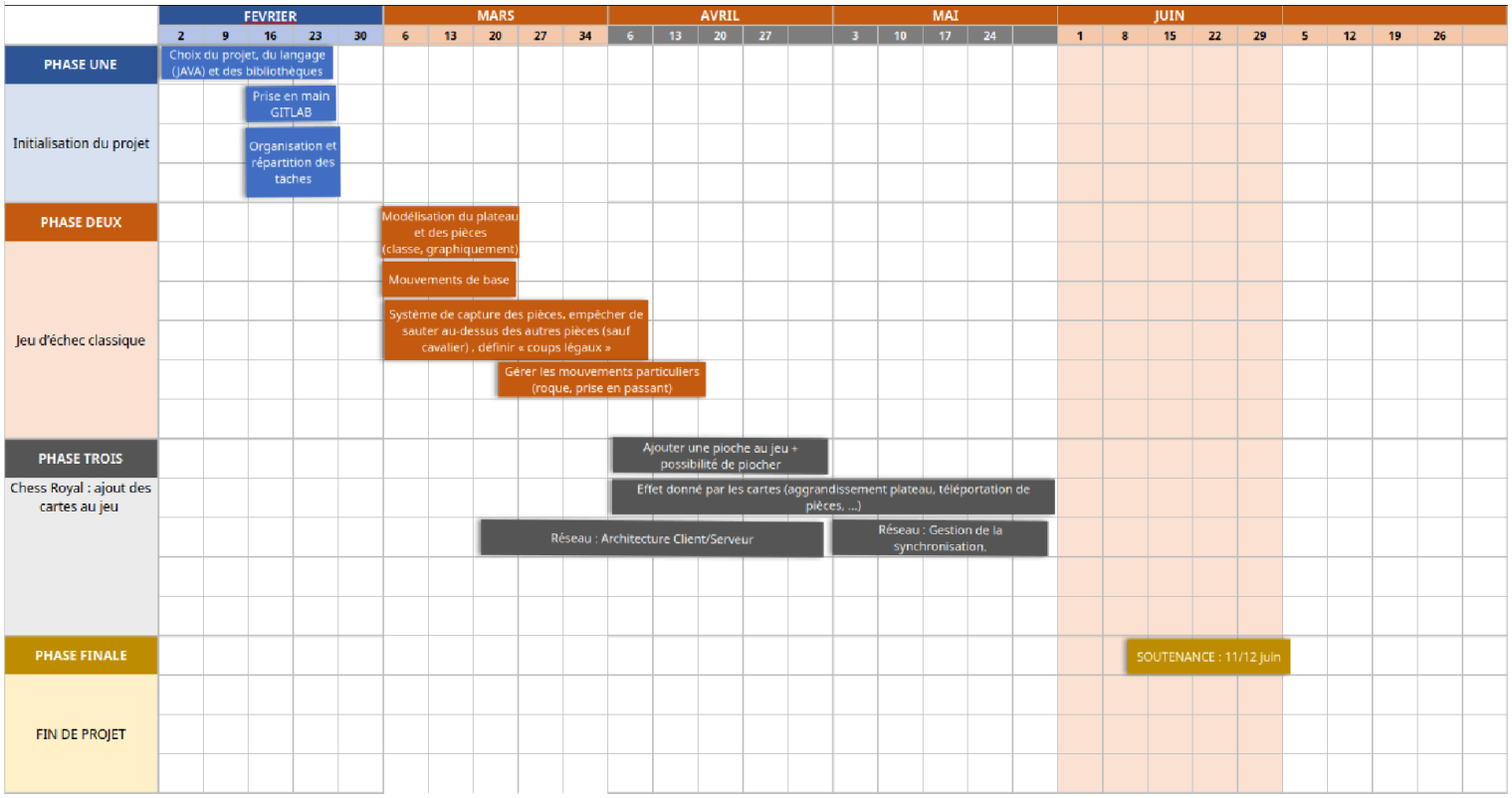
Quelque chose de particulièrement marquant est de voir le projet naître petit à petit. Au départ, il n'y avait que quelques idées, qui ont ensuite abouti à un jeu textuel, puis une interface graphique, du réseau, et enfin un jeu pratiquement complet, dont nous pouvons être fier.

La création de ChessRoyal aura finalement été l'occasion d'affiner mes compétences en programmation, en création de jeu, et surtout de découvrir ce qu'est la gestion d'un projet de grande envergure. Ce sont toutes des compétences que je suis content d'avoir acquis, et qui me serviront sans aucun doute à de nouvelles occasions.

Annexe A - Gestion de projet

A.1 Planning prévisionnel

Le planning prévisionnel a été organisé en quatre phases : initialisation du projet, jeu d'échecs classique, ajout des cartes et du réseau et finalisation. Il est illustré sous la forme d'un diagramme de Gantt couvrant toute la période de février à juin jusqu'à la soutenance du 12 juin 2026.



A.2 Plan de charge et suivit d'activité

Le plan de charge ci-dessous récapitule la répartition de l'effort (en heures), prévue au début du projet. Nous retrouvons les grandes activités du projet, composées des étapes nécessaires à l'imagination, la conception, le codage, l'intégration, et la présentation du jeu. Un total de 278 heures sont partagées par les cinq membres du groupe.

	Charge en H / Participant							Suivi d'activités (Charge consommée)						
Description de l'activité	**Charge en %**	**Charge en H**	**Arthur**	**Nathan**	**Ilyes**	**Liora**	**Solène**	**Charge en %**	**Charge en H**	**Arthur**	**Nathan**	**Ilyes**	**Liora**	**Solène**
Total	**100%**	**262**	**53**	**52**	**52**	**53**	**52**	**106%**	**278**	**56**	**58**	**53**	**58**	**53**
Gestion de projets	**17%**	**44**	**8**	**8**	**10**	**8**	**10**	**13%**	**34**	**6**	**6**	**8**	**6**	**8**
Réunion de lancement	2%	5	1	1	1	1	1	2%	5	1	1	1	1	1
Planning prévisionnel et Suivi d'activités	2%	5	1	1	1	1	1	2%	5	1	1	1	1	1
Réunions de suivi	8%	20	4	4	4	4	4	8%	20	4	4	4	4	4
Rédaction	5%	14	2	2	4	2	4	2%	4	0	0	2	0	2
Spécification	**4%**	**10**	**2**	**2**	**2**	**2**	**2**	**6%**	**15**	**3**	**3**	**3**	**3**	**3**
Définition des fonctionnalités	4%	10	2	2	2	2	2	6%	15	3	3	3	3	3
Conception préliminaire	**12%**	**32**	**6**	**7**	**6**	**7**	**6**	**16%**	**42**	**8**	**9**	**8**	**9**	**8**
Définition d'un modèle de données	3%	7	1	2	1	2	1	5%	12	2	3	2	3	2
Définition d'un format de fichiers associé	2%	5	1	1	1	1	1	2%	5	1	1	1	1	1
Définition des fonctionnalités au modèle de données	6%	15	3	3	3	3	3	8%	20	4	4	4	4	4
Définition des modules	2%	5	1	1	1	1	1	2%	5	1	1	1	1	1
Conception détaillée	**35%**	**91**	**20**	**18**	**17**	**19**	**17**	**40%**	**105**	**23**	**24**	**17**	**24**	**17**
Définition des classes	2%	6	1	1	1	2	1	2%	6	2	1	1	1	1
Définition des méthodes	8%	20	4	4	4	4	4	7%	18	4	4	3	4	3
Définition des tests unitaires	6%	15	3	3	3	3	3	6%	15	3	3	3	3	3
Auto-formation (réseaux)	18%	47	10	10	9	9	9	23%	59	13	13	10	13	10
Maquettage des interfaces	1%	3	2	0	0	1	0	3%	7	1	3	0	3	0
Codage	**19%**	**50**	**10**	**10**	**10**	**10**	**10**	**20%**	**53**	**11**	**11**	**10**	**11**	**10**
Codage des classes	2%	5	1	1	1	1	1	2%	5	1	1	1	1	1
Codage des méthodes	11%	30	6	6	6	6	6	13%	33	7	7	6	7	6
Codage des tests unitaires	6%	15	3	3	3	3	3	6%	15	3	3	3	3	3
Intégration	**6%**	**15**	**3**	**3**	**3**	**3**	**3**	**6%**	**15**	**3**	**3**	**3**	**3**	**3**
Intégration des modules	4%	10	2	2	2	2	2	4%	10	2	2	2	2	2
Tests d'intégration	2%	5	1	1	1	1	1	2%	5	1	1	1	1	1
Soutenance	**8%**	**20**	**4**	**4**	**4**	**4**	**4**	**5%**	**14**	**2**	**2**	**4**	**2**	**4**
Préparation de la soutenance	6%	15	3	3	3	3	3	3%	9	1	1	3	1	3
Soutenance	2%	5	1	1	1	1	1	2%	5	1	1	1	1	1

Annexe B - Code source

Code source de la classe Bishop.java qui est la classe de la piece “Fou”, qui montre la façon dont les pièces ont été codées en général.

```java
package models;

import java.util.ArrayList;

public class Bishop extends Piece{

    // Constructeur
    public Bishop(Position pos, int col) {
        super(pos, col);
        symbol = 'B';
    }

    // Méthode de déplacement
    public Position[] getPossibleMoves(Square[][] board) {

        ArrayList<Position> possibleMoves = new ArrayList<>();

        int currentRow = this.getRow();
        int currentCol = this.getCol();

        int[][] directions = {{-1, -1}, {-1, 1}, {1, 1}, {1, -1}}; // Directions disponibles

        for (int[] dir: directions) {
            int newRow = currentRow + dir[0];
            int newCol = currentCol + dir[1];

            while (newRow >= 0 && newRow < board.length && newCol >= 0 && newCol < board[0].length) {
                Square testedSquare = board[newRow][newCol]; // vérification pour les savoir quelles cases sont accessible
                // a partir de la case actuelle dans le while

                if (testedSquare == null) {
                    break;
                }

                if (testedSquare.getPiece() == null) {
                    possibleMoves.add(new Position(newRow, newCol));
                }

                else {
                    Piece targetPiece = testedSquare.getPiece();
                    if (targetPiece.getColor() != this.getColor()) {
                        possibleMoves.add(new Position(newRow, newCol));
                    }
                    break;
                }

                newRow += dir[0];
                newCol += dir[1];
            }

        }

        return possibleMoves.toArray(new Position[0]); // On stocke dans possibleMoves les cases accessibles
    }

}
```

Code source de la classe CardMessage.java qui décrit une action liée aux cartes à transmettre sur le réseau.

```java
package messages;

import java.io.Serializable;

///Pour transporter 3 infos : id de la carte (NUmCard), row et int = case ou on applique la carte
///
public class CardMessage implements Serializable {

    private static final long serialVersionUID = 1L;


    private final int NumCard; //Pour identifier la carte dans chessboardctrl
    private final int row;
    private final int col;

    public CardMessage(int NumCard, int row, int col) {
        this.NumCard = NumCard ;
        this.row = row;
        this.col = col;
    }

    public int getNumCard() { return NumCard; }
    public int getRow()              { return row; }
    public int getCol()              { return col; }

    @Override
    public String toString() {
        return "CardMessage[type=" + NumCard + " -> (" + row + "," + col + ")]";
    }
}
```